

МОДУЛЬ 4 · УРОК 4.3 · МИНИ-ПРОЕКТ

Игра «Угадай число»

Компьютер загадал число, а ты угадываешь! Соберём настоящую игру.

C++ с нуля · 45 минут

Компьютер **загадал число** от 0 до 99. Сколько попыток тебе нужно?

Сегодня соберём игру из `while`, `if` и `cin` — всё, что мы уже умеем.

Вспомним прошлый урок

- `while` и `for` повторяют тело по условию
- `if (условие) { ... }` — программа принимает решение
- `cin >> x;` — спрашиваем число у игрока

Соединим всё это в игру: повторяем попытки, пока не угадаешь.

План урока

- 1 Случайное число: `rand() % 100`
- 2 `break` — выйти из цикла досрочно
- 3 `continue` — пропустить шаг цикла
- 4 Собираем игру «Угадай число»
- 5 **Практика в классе** — улучшаем игру
- 6 **Домашнее задание**

Случайное число



main.cpp

```
#include <cstdlib>

int secret = rand() % 100;    // число от 0 до 99
```

`rand()` выдаёт случайное число, а `% 100` оставляет остаток 0–99.

`% 100` — это деление с остатком из модуля 2. Остаток всегда меньше 100.

Чтобы числа были разными

```
main.cpp

#include <cstdlib>
#include <ctime>

srand(time(0));           // «перемешать» один раз в начале
int secret = rand() % 100;
```

`srand(time(0))` запускают один раз в начале `main` — по часам.

Без него `rand()` каждый запуск выдаёт одно и то же число. С ним — каждый раз новое.

break · ВЫЙТИ ИЗ ЦИКЛА

`break` досрочно прерывает цикл — как стоп-кран.

```
main.cpp

while (true) {
    std::cin >> guess;
    if (guess == secret) {
        std::cout << "Угадал!\n";
        break;           // выходим из цикла
    }
}
```

7

Как только число угадано — `break` мгновенно выводит нас из цикла.

continue · пропустить шаг

`continue` пропускает остаток шага и переходит к следующему повтору.



```
main.cpp

for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;    // тройку пропускаем
    std::cout << i << " ";
}
```

Выведет **1 2 4 5** — на шаге `i == 3` тело пропустилось, цикл пошёл дальше.

break и continue · в чём разница

break

полностью **выходит** из цикла

continue

пропускает один шаг, цикл
продолжается

`break` — стоп всему циклу. `continue` — «этот шаг пропустим, дальше работаем».

Собираем игру · по шагам

Логика игры на пальцах:

- 1 Компьютер загадал `secret`
- 2 Игрок вводит число `guess`
- 3 Если `guess < secret` — «Больше!»
- 4 Если `guess > secret` — «Меньше!»
- 5 Если равны — «Угадал!» и `break`

Повторяем шаги 2–5 в цикле, пока не угадали.

Игра «Угадай число» целиком

```
main.cpp

srand(time(0));
int secret = rand() % 100;
int guess;
while (true) {
    std::cin >> guess;
    if (guess < secret) std::cout << "Больше!\n";
    else if (guess > secret) std::cout << "Меньше!\n";
    else { std::cout << "Угадал!\n"; break; }
}
```

11

Это вся игра! Нужны `#include <iostream>`, `<cstdlib>`, `<ctime>`.

Разбираем главное

```
main.cpp

while (true) {           // цикл «пока не угадаем»
    std::cin >> guess;    // игрок вводит число
    if (...) ...         // подсказка больше/меньше
    else { ...; break; }  // угадал — выходим
}
```

`while (true)` крутится вечно — выйти из него можно только через `break`.

Поэтому `break` тут обязателен: без него игра никогда не закончится.

Частые ошибки



Забыл break

`while (true)` без `break` —
игра не кончится.



Перепутал знаки

`<` и `>` местами —
подсказки «врут».



Забыл srand

Без `srand(time(0))` число
всегда одинаковое.

Главное в `while (true)` — не забыть `break`, иначе бесконечный цикл.

Проверь себя · что выведет программа? 🧠



main.cpp

```
for (int i = 1; i <= 5; i++) {  
    if (i == 2) continue;  
    std::cout << i << " ";  
}
```

Подумай 10 секунд...

Выведет **1 3 4 5**. На шаге `i == 2` сработал `continue` — двойку пропустили.

Проверь себя · break 🧠



main.cpp

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) break;  
    std::cout << i << " ";  
}
```

А здесь что появится?

Выведет **1 2**. На `i == 3` сработал `break` — цикл сразу остановился.

ПРАКТИКА В КЛАССЕ

Собираем «Угадай число»



Пишем настоящую игру и улучшаем её своими идеями.

♦ ЗАДАНИЕ

Задача 1 · Запускаем игру

Собери игру целиком и сыграй с соседом по парте:

```
main.cpp

srand(time(0));
int secret = rand() % 100;
int guess;
while (true) {
    std::cin >> guess;
    if (guess < secret) std::cout << "Больше!\n";
    else if (guess > secret) std::cout << "Меньше!\n";
    else { std::cout << "Угадал!\n"; break; }
}
```

✦ ЗАДАНИЕ

Задача 2 · Считаем попытки

Добавь в игру счётчик попыток. Заведи `int tries = 0;` до цикла и `tries++` на каждом вводе.

- Когда игрок угадал — выведи: «**Готово за ... попыток!**»
- Печатай `tries` в финальном сообщении

Подсказка: `tries++` ставим сразу после `cin`

✦ ЗАДАНИЕ

Задача 3 · Лёгкий уровень

Сделай версию полегче: компьютер загадывает число от **0 до 9** (`rand() % 10`).

- Поменяй только `% 100` на `% 10`
- Сыграй — угадывать стало проще?

Время: 6 минут

✦ ЗАДАНИЕ

Задача 4 · Ограничь попытки

★ со звёздочкой

Дай игроку только **5 попыток**. Используй `for (int i = 1; i <= 5; i++)` вместо `while (true)`.

- Угадал — выводим «Победа!» и `break`
- Попытки кончились — «Игра окончена, число было ...»

Кто добавит подсказку «осталось N попыток» — мастер!

Что мы сегодня научились 🧠

- `rand() % 100` — случайное число от 0 до 99
- `srand(time(0))` — чтобы числа были разными (один раз в начале)
- `break` — досрочно выйти из цикла
- `continue` — пропустить один шаг цикла
- Собрали игру из `while`, `if` и `cin` — всё уже знакомое!

Домашнее задание 🏠

1. Реши рабочий лист 4.3 — `break` / `continue` и своя версия игры.
2. Загляни в шпаргалку 4.3 — случайные числа и управление циклом.

Дальше: модуль 5 — функции и массивы: научимся хранить целые списки данных.